CMIS 320 Project 3

In this assignment you will create the physical database from your design created in Project 2. Project 2 was a logical model of the Acme Video Store database. We need to transition the logical model into a physical implementation. As noted in the course content, a physical implementation (or sometimes known as a physical model) is the database created on a system that we (or the users) will use. To this end, we use Oracle compliant SQL to create our database.

SQL is standard between many relational database types and is a part of Oracle, as noted in our readings this week. Also, SQL had a few different 'subsets' of language commands, which includes Data Definition Language (DDL) and Data Manipulation Language (DML), as two of the primary SQL language subsets (there are a few other language subsets).

To create the database in a database tool like Oracle, we use SQL's Data Definition Language (DDL), which allows us to tell the database how the tables (entities) and the associated fields within the tables (attributes) are described.

Step One: CREATE THE SQL DDL FOR YOUR DATABASE

For your logical database model created in Project 2, create the SQL DDL commands that will be submitted to Oracle. Oracle will create the data structures to hold your data.

EXAMPLE:

- From our reading - Creating a table
- Describe in detail the layout of each table in the database
- Use CREATE TABLE statement
- The word 'TABLE' in that statement is followed by the table name
- Follow this with the names and built-in Oracle datatypes of the attributes in the table
- Datatypes define type and size of data
    - Make sure that we do not use deprecated data types

- Table and column name restrictions
    - Names cannot exceed 30 characters
    - Both must start with a letter
    - Can contain letters, numbers, and underscores (_)
    - Cannot contain spaces
    - Identifying attribute names must always be meaningful

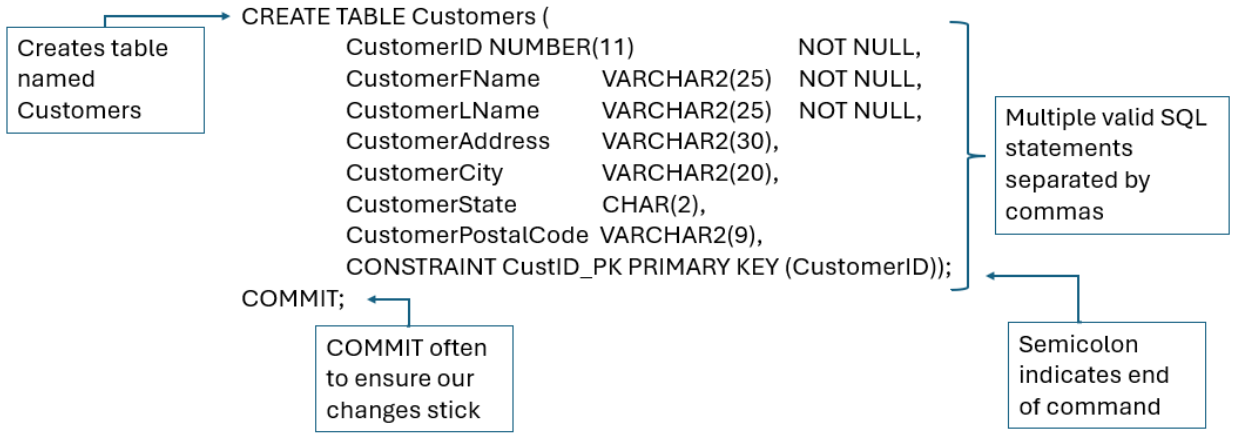


Figure 1: Create table REP using DDL

In addition to reviewing the course material in this week's content area, this resource might be helpful:

- Introduction to Oracle CREATE TABLE: https://www.oracletutorial.com/oraclebasics/oracle-create-table/

If an error is encountered or you wish to re-create the table, you will first need to DROP the existing table. Dropping a table is done when:

- Correcting errors by dropping (deleting) a table and starting over
- Useful when table is created before errors are discovered
- Command is followed by the table to be dropped and a semicolon
- Any data in table also deleted

Be sure to SAVE your work. Saving your work with a .sql extension will:

- Allow you to use commands again without retyping
- Save commands in a script file or script

- This will basically be a text file with an .sql extension

Step Two: LOAD DATA INTO YOUR DATABASE

Following the creation of the database structure (the tables and fields) using DDL, we'll need to load data into these structures (e.g., populate data into the tables). Populating the Acme Video Store database is done by using Data Manipulation Language (DML) SQL commands.

As an example, let's create a table not related to our project. We will use INSERT to add the first few records to it. The example code below will create a table named users that has 5 columns. We'll have a userid column that will be the PRIMARY KEY (the column that will always have unique values and allow us to uniquely identify a row), and then the name, age, state, and email columns. The SQL DDL is:

```
CREATE TABLE users (
   USERID          INTEGER PRIMARY KEY,
   NAME            VARCHAR2(100),
   AGE             NUMBER,
   STATE           CHAR(2),
   EMAIL           VARCHAR2(100)
);
```

Load data into the 'users' table by using the SQL DML INSERT command. We will add the user Paul with a userid of 1, an age of 24, from the state of Michigan, and with an email address of paul@example.com using the query below:

```
INSERT INTO users VALUES (1, "Paul", 24, "Michigan", "paul@example.com");
```

Let's add a couple more records into that table:

```
INSERT INTO users (userid, name, state) VALUES (2, "Molly", "New Jersey");
INSERT INTO users (userid, name, state, age) VALUES (3,"Robert", "New York", 19);
```

In this case, the first value is assigned to the first mentioned column, so "Molly" is assigned to the name column, and "New Jersey" to the state column. Then for the other record, the column name is given the value of "Robert", the column state gets "New York", the column age is assigned 19.

As noted above, the process of creating the database structure by using DDL followed by using the DML Insert command will be similar to creating the database structure and loading data from Project 2.

In addition, this resource may be helpful:

- Oracle Insert Table: https://www.oracletutorial.com/oracle-basics/oracle-insert/

Step Three: WRITE SQL QUERIES TO RETRIEVE DATA FROM THE DATABASE

Using SQL, we can interact data from the database by writing queries. Queries are a part of SQL. Here's what an example query looks like, using the SELECT statement:

```
SELECT *
FROM users;
```

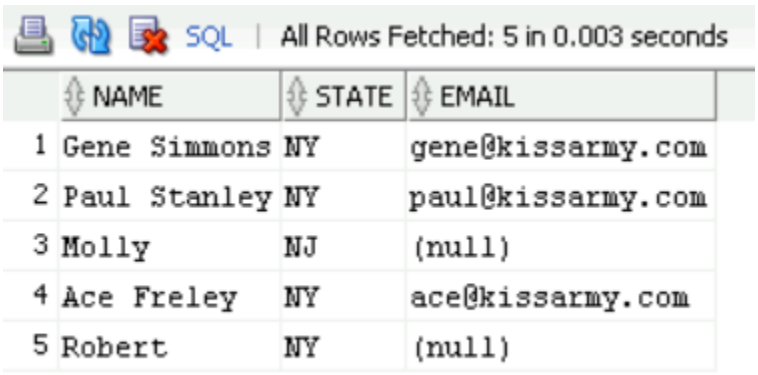
Using this SELECT statement, the query selects all the data from all the columns in the customer's table and returns data like so:

	USERID	NAME	AGE	STATE	EMAIL
1	1	Gene Simmons	71	NY	gene@kissarmy.com
2	3	Paul Stanley	72	NY	paul@kissarmy.com
3	2	Molly	(null)	NJ	(null)
4	4	Ace Freley	70	NY	ace@kissarmy.com
5	5	Robert	19	NY	(null)

Figure 2: SELECT statement results from the users table

The asterisk wildcard character (*) refers to "all" and selects all the rows and columns. We can replace it with specific column names instead — here only those columns will be returned by the query

```
SELECT name, state, email
FROM users;
```

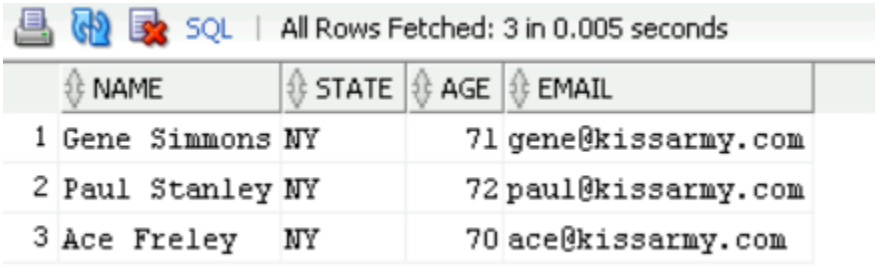


SQL | All Rows Fetched: 5 in 0.003 seconds

	NAME	STATE	EMAIL
1	Gene Simmons	NY	gene@kissarmy.com
2	Paul Stanley	NY	paul@kissarmy.com
3	Molly	NJ	(null)
4	Ace Freley	NY	ace@kissarmy.com
5	Robert	NY	(null)

Adding a WHERE clause allows you to filter what gets returned:

```
SELECT name, state, age, email
FROM users
WHERE age >= 60;
```



SQL | All Rows Fetched: 3 in 0.005 seconds

	NAME	STATE	AGE	EMAIL
1	Gene Simmons	NY	71	gene@kissarmy.com
2	Paul Stanley	NY	72	paul@kissarmy.com
3	Ace Freley	NY	70	ace@kissarmy.com

This query only shows those users with an age value greater than or equal to 60. This query returns all data from the products table with an age value of greater than 30. The use of ORDER BY keyword just means the results will be ordered using the age column from the lowest value to the highest. Of course, this brief (and limited) example is explained in greater detail in the course content.

Assignment Requirements:

1. Step One: Write the first SQL script to create Oracle database tables using SQL Data Definition Language (DDL) for each table listed in the metadata of Project 2.
    a. Include DROP TABLE statements for each table at the script.
    b. Ensure that referential integrity is established between related tables. Every table must have a Primary Key. Related tables must have Foreign Keys defined. These can be defined in ALTER TABLE statements later in the script.
    c. At least one ALTER TABLE statement must be included in the first script.
    d. Ensure all table constraints are properly defined such as PRIMARY KEYS, FOREIGN KEYS, etc.
    e. Be sure to save your first SQL script that creates all of your tables with a .sql extension.
2. Step Two: Write the second SQL script to populate all the previously defined tables with at least five records in each table. Some may need more. For instance, the table that holds our historical rentals will likely have far more than five records.
    a. Include DELETE TABLE statements for each table at the top of the script.
    b. Ensure the syntax is correct for numerical or character-based values being inserted.
    c. Ensure that the related Primary Keys exist prior to inserting them into the related tables as Foreign Keys.
    d. Be sure to save your second SQL script that populates all of your tables with a .sql extension.
3. Step Three: Write a third SQL script that performs queries against your database:
    a. After composing your queries, run the SQL script until no errors are encountered:
        1. Retrieve all of your customers' names, account numbers, and addresses (street and zip code only), sorted by account number.
        2. Retrieve all of the videos rented in the last 30 days and sort in chronological rental date order.
        3. Produce a list of your distributors and all their information sorted in order by company name.
        4. Update a customer name to change their maiden name to a married name. You can choose which row to update. Make sure that you use the primary key column in your WHERE clause to affect only a specific row. You may want to include a ROLLBACK statement to undo your data update.
        5. Delete a customer from the database. You can choose which row to delete. Make sure that you use the primary key column in your WHERE clause to affect only a specific row. You may want to include a ROLLBACK statement to undo your data deletion.
    b. Be sure to save your third SQL script with the queries defined above with a .sql extension.

What to Submit for Grading?

1. You should have three .sql scripts that must be submitted.
2. Use the proper naming conventions defined in the submission requirements.
3. Do not submit more than these three files

Grading rubric

Attribute	Meets expectations	Does not meet expectations
CREATE TABLE and ALTER TABLE SQL statements	30 points All SQL statements are syntactically correct and execute without errors; all integrity constraints are properly declared	0 points Many SQL statements fail due to syntax errors or SQL is missing
INSERT SQL statements	25 points All SQL statements are syntactically correct and execute without errors.	0 points Many SQL statements fail due to syntax errors and/or integrity constraint violations or SQL is missing
SELECT SQL statements	20 points All SQL statements are syntactically correct and execute without errors.	0 points Many SQL statements fail due to syntax errors or SQL is missing
UPDATE and DELETE SQL statements	10 points Statements execute without errors based on primary key column in WHERE clause	0 points Statements fail due to syntax or other errors
SQL script files	15 points Demonstrates full ability to create and use an Oracle SQL script filed	0 points Many errors setting up and using an SQL script file or no attempt made at all

Additional resources that you might use:

- Oracle Database Basics: https://www.oracletutorial.com/oracle-basics/
- Oracle Data Definition Language (DDL) description: https://docs.oracle.com/cd/B14117_01/server.101/b10759/statements_1001.htm
- Oracle Insert Table: https://www.oracletutorial.com/oracle-basics/oracle-insert/
- Oracle DROP Table: https://www.oracletutorial.com/oracle-basics/oracle-drop-table/
- Introduction to Oracle CREATE TABLE: https://www.oracletutorial.com/oraclebasics/oracle-create-table/